



ΗΥ340 : ΓΛΩΣΣΕΣ ΚΑΙ ΜΕΤΑΦΡΑΣΤΕΣ

ΠΑΝΕΠΙΣΤΗΜΙΟ ΚΡΗΤΗΣ,
ΣΧΟΛΗ ΘΕΤΙΚΩΝ ΕΠΙΣΤΗΜΩΝ,
ΤΜΗΜΑ ΕΠΙΣΤΗΜΗΣ ΥΠΟΛΟΓΙΣΤΩΝ

```
VAR i:Integer;  
    FUNCTION(Symbol) replicate  
        x = (function(x,y){return x+y;});  
        class DelFunctor: public std::unary_function<
```

ΔΙΔΑΣΚΩΝ

Αντώνιος Σαββίδης



HY340 : ΓΛΩΣΣΕΣ ΚΑΙ ΜΕΤΑΦΡΑΣΤΕΣ

Διάλεξη 3η ΛΕΞΙΚΟΓΡΑΦΙΚΗ ΑΝΑΛΥΣΗ - II



Περιεχόμενα

- *Αιτιοκρατικά αυτόματα - DFA*
- Αλγόριθμος μετατροπής NFA $\rightarrow$ DFA
- Γεννήτριες λεξικογραφικών αναλυτών
- Το πρόγραμμα *Lex*



Αιτιοκρατικά αυτόματα (1/5)

- Ένα αιτιοκρατικό αυτόματο (deterministic finite automaton – DFA) είναι μία **ειδική περίπτωση ενός *NFA***, με δύο βασικούς περιορισμούς
 - **Δεν επιτρέπονται κενές μεταβάσεις** μέσω του συμβόλου ϵ
 - ◆ δηλαδή την κενή λέξη - empty string
 - Για κάθε κατάσταση s και σύμβολο εισόδου a , υπάρχει το πολύ μία κατάσταση στο σύνολο ***move(s, a)***
 - ◆ δηλαδή από μία κατάσταση ορίζεται το πολύ μία μετάβαση για κάθε ξεχωριστό σύμβολο εισόδου

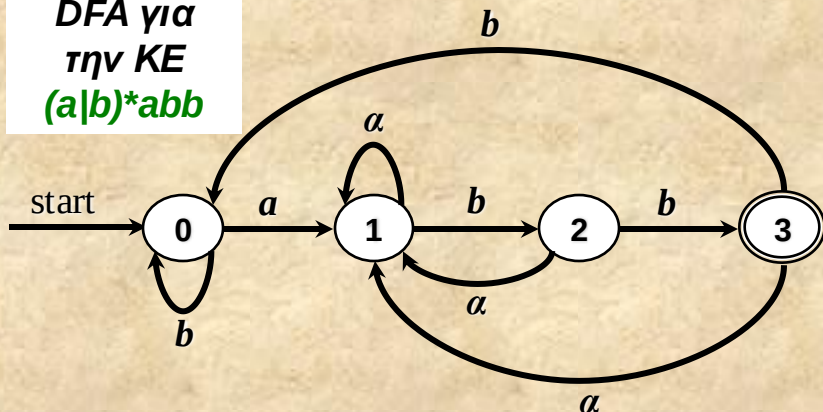


Αιτιοκρατικά αυτόματα (2/5)

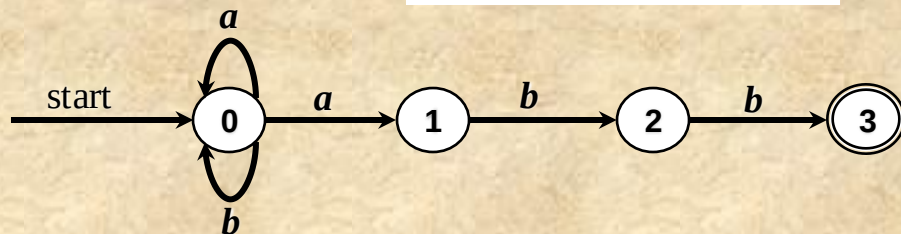
- Διαισθητικά γίνεται αντιληπτό ότι ένα DFA είναι λιγότερο «ευέλικτο» από ένα NFA
 - με την έννοια ότι για την δημιουργία ενός DFA που «κάνει» ότι ακριβώς ένα αντίστοιχο NFA συνήθως θα χρειαστούμε περισσότερες καταστάσεις
 - ◆ η αλλιώς ένα «μεγαλύτερο» αυτόματο
 - θα δούμε ένα DFA το οποίο αναγνωρίζει την κανονική γλώσσα που ορίζεται από την κανονική έκφραση **$(a|b)^*abb$**
 - ◆ Για την οποία έχουμε ήδη παράγει ένα NFA με τον αλγόριθμο παραγωγής ενός NFA από μία κανονική έκφραση

Αιτιοκρατικά αυτόματα (3/5)

DFA για
την ΚΕ
 $(a|b)^*abb$



Handmade NFA για
την ΚΕ $(a|b)^*abb$



- Εδώ φαίνεται ότι το DFA έχει πολύ λιγότερα states (συνολικά 4) από το NFA που δημιουργήσαμε μηχανιστικά (συνολικά 11)
 - Σε αυτό φταίει ότι ο αλγόριθμος $RE \rightarrow NFA$ δεν δημιουργεί αυτόματο με ελαχιστοποίηση καταστάσεων
- Όμως, αφού έχουμε κατασκευάσει ένα NFA, γιατί δεν κάνουμε εξομοίωση της λειτουργίας του απευθείας αντί να ασχολούμαστε με μετατροπή σε DFA?

Αιτιοκρατικά αυτόματα (4/5)

- Η απάντησή είναι ότι η συνύπαρξη
 - τόσο των «κενών» ε μεταβάσεων
 - όσο και των πολλαπλών πιθανών μεταβάσεων από οποιαδήποτε κατάσταση για ένα σύμβολο εισόδου
- ➔ *καθιστούν την εξομοίωση σε ιδιαίτερα δύσκολη κατασκευαστικά και χρονοβόρα εκτελεστικά διαδικασία*
 - ➔ ιδιαίτερα απαιτητική σε μνήμη για την αποθήκευση των εναλλακτικών μεταβάσεων
 - ➔ αλλά και σε ταχύτητα για την εκτέλεση των εναλλακτικών μεταβάσεων
- Αντιθέτως η εξομοίωση της λειτουργίας ενός DFA είναι πάρα πολύ απλή και υλοποιείται ιδιαίτερα εύκολα από τον παρακάτω αλγόριθμο... ➔



Αιτιοκρατικά αυτόματα (5/5)

Είσοδος. Μία λέξη εισόδου x που τερματίζεται από έναν ειδικό χαρακτήρα (end-of-string) **eof**. Ένα DFA D με αρχική κατάσταση s_0 , συνάρτηση μετάβασης $move(s, c)$, και τελικές καταστάσεις το σύνολο F .

Έξοδος. “yes” εάν το D δέχεται την λέξη x , αλλιώς “no”.

Μέθοδος. Εφαρμόζεται ακριβώς ο παρακάτω αλγόριθμος:

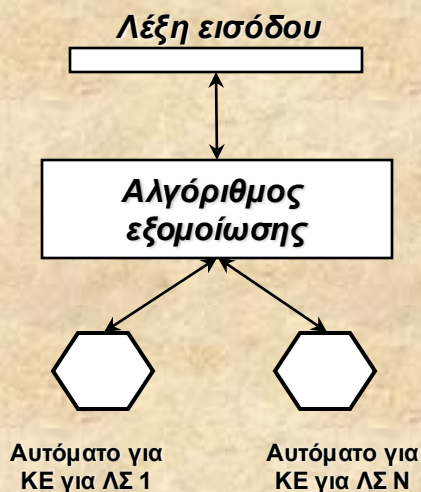
```
 $s = s_0;$ 
 $c = nextchar();$ 
while  $c \neq eof$  do
  begin
     $s = move(s, c);$ 
     $c = nextchar();$ 
    if  $s = no-viable-state$  then
      return “no”;
  end
  if  $s$  in  $F$  then
    return “yes”;
else
  return “no”;
```

• Ένα ιδιαίτερα θετικό χαρακτηριστικό του αλγόριθμου εξομοίωσης είναι ότι η μετατροπή του ώστε να εξομοιώνονται παράλληλα περισσότερα από ένα αυτόματα είναι τετριμμένη.

• Έτσι, θα μπορούσαμε να έχουμε N regular expressions για N tokens, με τα αντίστοιχα DFAs, και να πετυχαίνουμε αναγνώριση του προτύπου λέξης εισόδου με «παράλληλη» λειτουργία των αυτομάτων.

• Επίσης μπορούμε να εντοπίζουμε γρήγορα εάν η λέξη εισόδου δεν μπορεί να αναγνωριστεί ως λεξικογραφικό στοιχείο για την περίπτωση που δεν ορίζεται μετάβαση για τον χαρακτήρα εισόδου.

Ένθετο - Αιτιοκρατικά αυτόματα



Αυτή η μέθοδος συνιστά μία επιπλέον τακτική κατασκευής λεξικογραφικού αναλυτή σε κώδικα

Έστω $move(t, s, c)$ η οποία εφαρμόζει μετάβαση από την κατάσταση s με σύμβολο c βάσει του πίνακα μετάβασης t και επιστρέφει τη νέα κατάσταση.

```

S = [S0L, S0I, ..., S0K];
c = nextchar();
while c ≠ eof do
begin
    S = [move(t0, S[0], c), ..., move(tk, S[k], c)];
    c = nextchar();
end

```

```

if ∃j: s[j] in F of tj then .....> Εάν έστω και ένα
    return j;                          αυτόματο είναι σε
else                                    τελική κατάσταση
    return -1;

```



Περιεχόμενα

- Αιτιοκρατικά αυτόματα - DFA
- *Αλγόριθμος μετατροπής NFA → DFA*
- Γεννήτριες λεξικογραφικών αναλυτών
- Το πρόγραμμα Lex



Αλγόριθμος μετατροπής ***NFA*** $\rightarrow$ ***DFA*** (1/11)

- Θα παρουσιάσουμε έναν αλγόριθμο που παράγει ένα DFA με καταστάσεις οι οποίες αντιστοιχούν σε ***υποσύνολα των καταστάσεων του αρχικού NFA***
 - Για το λόγο αυτό ο αλγόριθμος είναι γνωστός ως «μέθοδος με κατασκευή υποσυνόλων» - ***subset construction method***
- Παράγεται ένα DFA τέτοιο ώστε μετά την ανάγνωση της εισόδου $\alpha_1 \alpha_2 \dots \alpha_n$
 - το DFA βρίσκεται σε μία κατάσταση που αναπαριστά το ακριβές υποσύνολο ***T*** όλων των δυνατών καταστάσεων στις οποίες το αρχικό NFA μπορεί να εισέλθει με την ίδια ακριβώς είσοδο



Αλγόριθμος μετατροπής $NFA \rightarrow DFA$ (2/11)

Για τον αλγόριθμο θα ορίσουμε ορισμένες απλές και χρήσιμες λειτουργίες (συναρτήσεις) για τις καταστάσεις του NFA

ΛΕΙΤΟΥΡΓΙΑ	ΠΕΡΙΓΡΑΦΗ
$\varepsilon\text{-closure}(s)$	Το σύνολο των καταστάσεων στις οποίες το NFA μπορεί να εισέλθει από την κατάσταση s αποκλειστικά με μεταβάσεις ε .
$\varepsilon\text{-closure}(T)$	Το σύνολο των καταστάσεων στις οποίες το NFA μπορεί να εισέλθει από καταστάσεις s του T αποκλειστικά με μεταβάσεις ε .
$move(T, a)$	Το σύνολο των καταστάσεων στις οποίες το NFA μπορεί να εισέλθει από καταστάσεις s του T για το σύμβολο εισόδου a .

ΠΑΡΑΤΗΡΗΣΕΙΣ

- Πριν ακόμη γίνει η ανάγνωση του πρώτου συμβόλου από τη λέξη εισόδου, παρατηρούμε ότι ένα NFA μπορεί ήδη να βρίσκεται σε οποιαδήποτε από τις καταστάσεις του $\varepsilon\text{-closure}(s_0)$, με s_0 την αρχική κατάσταση.
- Τώρα, έστω ότι T είναι ακριβώς το σύνολο των καταστάσεων στις οποίες μπορεί να εισέλθει το NFA αφού γίνει ανάγνωση μίας δεδομένης ακολουθίας συμβόλων εισόδου και έστω a να είναι το επόμενο σύμβολο προς ανάγνωση.
- Μετά την ανάγνωση του a , το NFA μπορεί να βρίσκεται σε οποιαδήποτε από τις καταστάσεις του $T' = move(T, a)$. Επιπλέον, λόγω των κενών μεταβάσεων, το NFA μπορεί αυτομάτως να εισέλθει σε οποιαδήποτε από τις καταστάσεις του $\varepsilon\text{-closure}(T')$.



Αλγόριθμος μετατροπής $NFA \rightarrow DFA$ (3/11)

ΕΠΕΞΗΓΗΣΕΙΣ

Κατασκευάζουμε το σύνολο $Dstates$, δηλ. το σύνολο των καταστάσεων του D , και τον $Dtran$, δηλ. τον πίνακα μετάβασης του D ως εξής:

- Κάθε κατάσταση του D αντιστοιχεί σε έναν σύνολο από καταστάσεις στις οποίες το N μπορεί να εισέλθει μετά την ανάγνωση ακολουθίας συμβόλων εισόδου, συμπεριλαμβανομένων των *ε-μεταβάσεων*.

- Η αρχική κατάσταση του D είναι το *ε-closure*(s_0).

- Καταστάσεις και μεταβάσεις προστίθενται στο D χρησιμοποιώντας τον διπλανό αλγόριθμο.

- Μία κατάσταση του D είναι τελική εφόσον περιέχει (καθώς είναι σύνολο από καταστάσεις του N) τουλάχιστον μία τελική κατάσταση του N .

Είσοδος. Ένα NFA N .

Έξοδος. Ένα DFA D που αναγνωρίζει την ίδια γλώσσα.

Μέθοδος. Διαδοχική κατασκευή του πίνακα μετάβασης $DTran$ του D . Κάθε κατάσταση του DFA είναι ένα σύνολο από καταστάσεις του NFA, ενώ κατασκευάζουμε τον $DTran$ ώστε οι μεταβάσεις του D να εξομοιώνουν ταυτόχρονα όλες τις πιθανές μεταβάσεις του N για δεδομένη λέξη εισόδου.

$Dstates = \{T_0\} : T_0 = \varepsilon\text{-closure}(s_0)$ και T_0 unmarked;

while $\exists T \in Dstates : T$ unmarked **do**

begin

$mark(T)$;

for $\forall$ σύμβολο εισόδου a **do**

begin

$U = \varepsilon\text{-closure}(\text{move}(T, a))$;

if $U \notin Dstates$ **then**

 Πρόσθεσε το unmarked U στο $Dstates$;

$DTran[T, a] = U$;

end

end



Αλγόριθμος μετατροπής *NFA* $\rightarrow$ *DFA* (4/11)

ΕΠΕΞΗΓΗΣΕΙΣ

Ο υπολογισμός του ϵ -closure είναι μία τυπική διαδικασία αναζήτησης (η συγκεκριμένη είναι depth first) σε γράφο κόμβων που είναι προσιτοί από ένα δεδομένο σύνολο κόμβων. Στην περίπτωση μας:

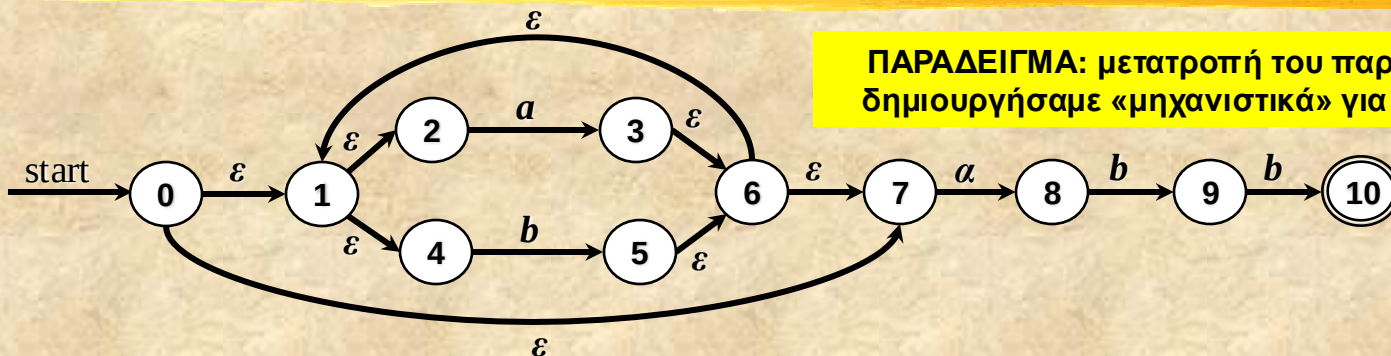
- Οι καταστάσεις του T είναι το αρχικά δεδομένο σύνολο από κόμβους, ενώ ο υποκείμενος γράφος περιέχει όλες τις ϵ -μεταβάσεις του αρχικού NFA.

- Ο διπλανός αλγόριθμος υπολογισμού του ϵ -closure ουσιαστικά υλοποιεί αναζήτηση κόμβων σε γράφο, χρησιμοποιώντας μία στοίβα όπου κρατούνται οι καταστάσεις των οποίων οι ακμές προς άλλες καταστάσεις δεν έχουν ακόμη ελεγχθεί για την ύπαρξη ϵ -μεταβάσεων.

```
 $\epsilon$ -closure( $T$ )  
begin  
   $A = \{\}$ ;  
   $\forall s \in T$ : push(stack,  $s$ );  
  while not empty(stack) do  
    begin  
       $t = \text{pop}(\text{stack})$ ;  
      for  $\forall u: \exists \epsilon$ -μετάβαση  $t \rightarrow u$  do  
        if  $u \notin A$  then  
          begin  
             $A = A \cup \{u\}$ ;  
            push(stack,  $u$ );  
          end  
        end  
      end  
    end  
  return  $A$ ;  
end
```



Αλγόριθμος μετατροπής $NFA \rightarrow DFA$ (5/11)



ΠΑΡΑΔΕΙΓΜΑ: μετατροπή του παρακάτω NFA, το οποίο δημιουργήσαμε «μηχανιστικά» για την $(a|b)^*abb$, σε DFA

$Dstates = \{T_0\} : T_0 = \epsilon\text{-closure}(s_0)$ και T_0 unmarked; 

while $\exists T \in Dstates : T$ unmarked **do**

begin

$mark(T)$;

for $\forall$ σύμβολο εισόδου a **do**

begin

$U = \epsilon\text{-closure}(\text{move}(T, a))$;

if $U \notin Dstates$ **then**

 Πρόσθεσε το unmarked U στο $Dstates$;

$DTran[T, a] = U$;

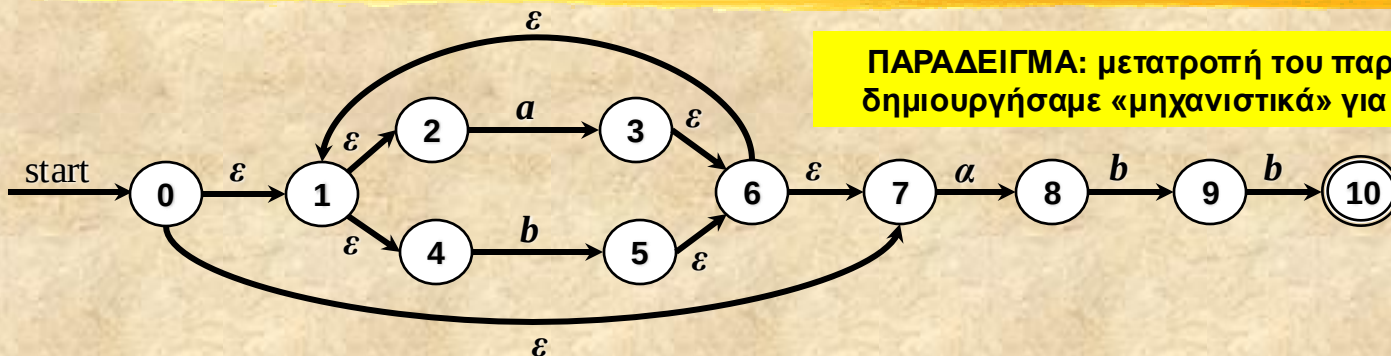
end

end

1. $Dstates = \epsilon\text{-closure}(0)$, το οποίο είναι $A = \{0, 1, 2, 4, 7\}$



Αλγόριθμος μετατροπής $NFA \rightarrow DFA$ (6/11)



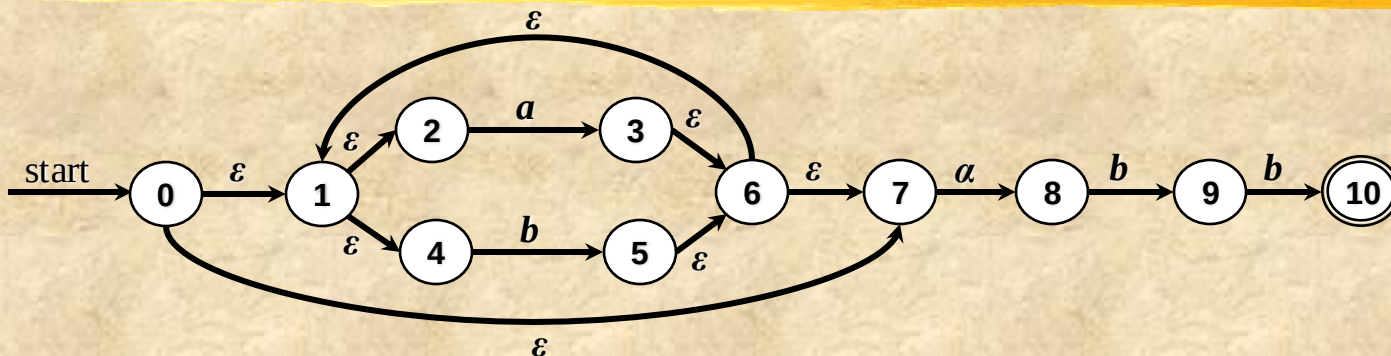
ΠΑΡΑΔΕΙΓΜΑ: μετατροπή του παρακάτω NFA, το οποίο δημιουργήσαμε «μηχανιστικά» για την $(a|b)^*abb$, σε DFA

```
Dstates = { $T_0$ } :  $T_0 = \epsilon$ -closure( $s_0$ ) και  $T_0$  unmarked;  
while  $\exists T \in Dstates$ :  $T$  unmarked do  
  begin  
    mark( $T$ );  
    for  $\forall$  σύμβολο εισόδου  $a$  do   
      begin  
         $U = \epsilon$ -closure(move( $T, a$ ));  
        if  $U \notin Dstates$  then  
          Πρόσθεσε το unmarked  $U$  στο  $Dstates$ ;  
           $DTran[T, a] = U$ ;  
        end  
      end  
    end  
  end
```

2. **mark(A)**, ενώ έπειτα αρχίζουμε μία ανακύκλωση για κάθε σύμβολο εισόδου (δηλ. σύμβολο του αλφαβήτου), που είναι τα $\{a, b\}$.



Αλγόριθμος μετατροπής $NFA \rightarrow DFA$ (7/11)



$Dstates = \{T_0\} : T_0 = \epsilon\text{-closure}(s_0)$ και T_0 unmarked;

while $\exists T \in Dstates : T$ unmarked **do**
begin

$mark(T)$;

for $\forall$ σύμβολο εισόδου a **do**

begin

$U = \epsilon\text{-closure}(\text{move}(T, a))$; 

if $U \notin Dstates$ **then**

 Πρόσθεσε το unmarked U στο $Dstates$;

$DTran[T, a] = U$;

end

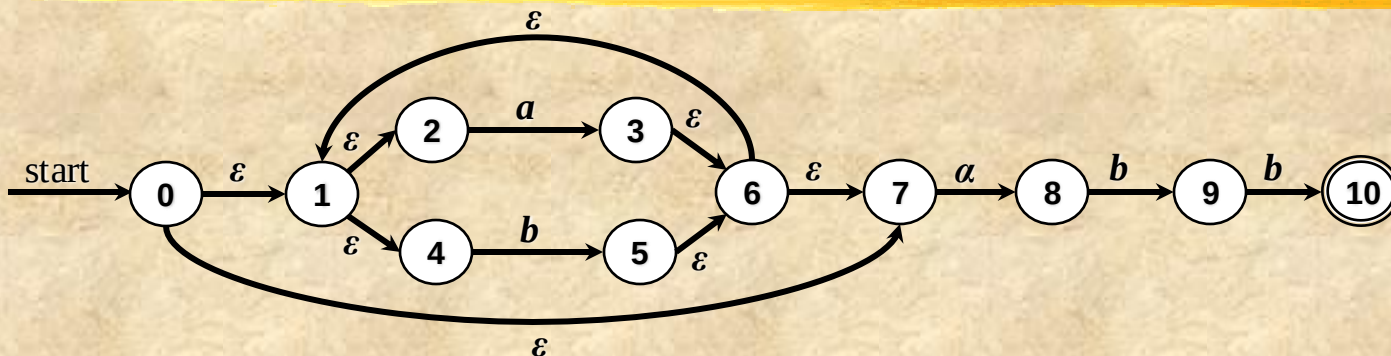
end

3. Για τον υπολογισμό του $\epsilon\text{-closure}(\text{move}(T, a))$ πρέπει να υπολογίσουμε το $\text{move}(T, a)$, με T να είναι το $A = \{0, 1, 2, 4, 7\}$.

Το $\text{move}(T, a) = \{3, 8\}$, και έτσι το $\epsilon\text{-closure}(\text{move}(T, a))$ προκύπτει να είναι σύνολο $B = \{1, 2, 3, 4, 6, 7, 8\}$



Αλγόριθμος μετατροπής $NFA \rightarrow DFA$ (8/11)



$Dstates = \{T_0\} : T_0 = \varepsilon\text{-closure}(s_0)$ και T_0 unmarked;

while $\exists T \in Dstates : T$ unmarked **do**

begin

$mark(T)$;

for $\forall$ σύμβολο εισόδου a **do**

begin

$U = \varepsilon\text{-closure}(\text{move}(T, a))$;

if $U \notin Dstates$ **then**

 Πρόσθεσε το unmarked U στο $Dstates$;

$DTran[T, a] = U$; 

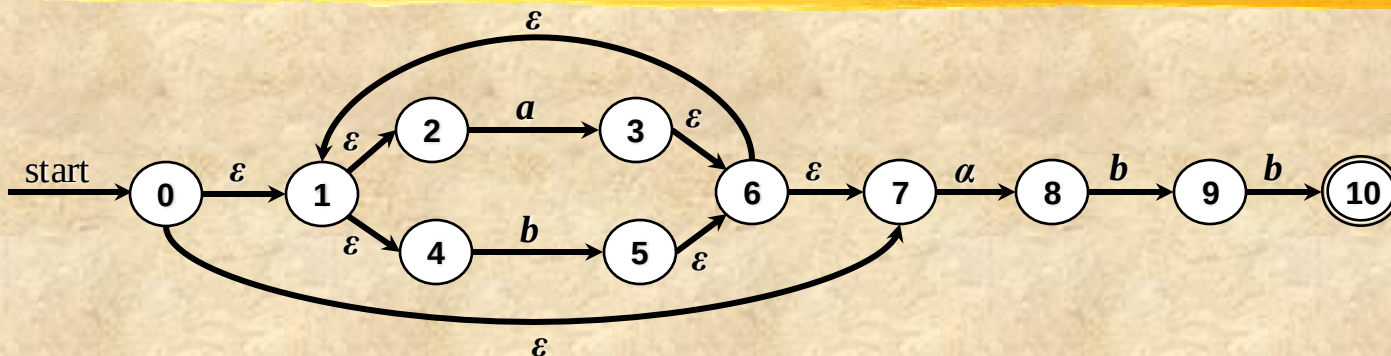
end

end

4. Καθώς το B δεν είναι στο $Dstates$,
τώρα έχουμε $Dstates = \{A, B\}$. Έπειτα
έχουμε $Dtran[A, a] = B$



Αλγόριθμος μετατροπής $NFA \rightarrow DFA$ (9/11)



```

Dstates = {T0} : T0 = ε-closure(s0) και T0 unmarked;
while ∃ T ∈ Dstates: T unmarked do
begin
  mark(T);
  for ∀ σύμβολο εισόδου a do
  begin
    U = ε-closure(move(T, a));
    if U ∉ Dstates then
      Πρόσθεσε το unmarked U στο Dstates;
      DTran[T, a] = U;
    end
  end
end

```

```

Dstates = {T0} : T0 = ε-closure(s0) και T0 unmarked;
while ∃ T ∈ Dstates: T unmarked do
begin
  mark(T);
  for ∀ σύμβολο εισόδου a do
  begin
    U = ε-closure(move(T, a));
    if U ∉ Dstates then
      Πρόσθεσε το unmarked U στο Dstates;
      DTran[T, a] = U;
    end
  end
end

```

5. Ομοίως για τον υπολογισμό του ε -closure(move(T,b)) πρέπει να υπολογίσουμε το move(T,b), με T να είναι το $A = \{0, 1, 2, 4, 7\}$. Το $\text{move}(T,b) = \{5\}$, και έτσι το $\varepsilon\text{-closure}(\text{move}(T,b))$ προκύπτει να είναι σύνολο $\Gamma = \{1, 2, 4, 5, 6, 7\}$

6. Παρομοίως, καθώς το Γ δεν είναι στο Dstates, τώρα έχουμε $Dstates = \{A, B, \Gamma\}$. Έπειτα έχουμε $Dtran[A, b] = \Gamma$



Αλγόριθμος μετατροπής $NFA \rightarrow DFA$ (10/11)

• Συνεχίζοντας τη διαδικασία αυτή με τα σύνολα B και Γ τα οποία είναι unmarked, φτάνουμε σε ένα σημείο όπου δεν υπάρχουν άλλα σύνολα από καταστάσεις του αρχικού NFA που να μην είναι marked.

• Ο τερματισμός αυτού του αλγορίθμου είναι εξασφαλισμένος καθώς ο αριθμός των διαφορετικών υποσυνόλων από καταστάσεις του συνόλου των καταστάσεων του NFA είναι πεπερασμένος (για την ακρίβεια $2^{11} - 1$, $n=11$ κόμβοι, δηλ. 2^n πλην το $\emptyset$), και κάθε σύνολο όταν γίνει marked δεν εμπλέκεται ποτέ αργότερα στην επεξεργασία.

• Βέβαια, είναι προφανές ότι ο αλγόριθμος αυτός είναι δυνατό να οδηγήσει σε εκθετικό αριθμό από καταστάσεις για το παραγόμενο DFA, ως προς το αρχικό αριθμό καταστάσεων του NFA, ωστόσο αυτό αυτό δεν είναι πιθανό.

Τα σύνολα που παράγονται τελικά για το συγκεκριμένο αυτόματο, με καθένα να ορίζει και μία ξεχωριστή κατάσταση του D, είναι τα παρακάτω, ενώ δίπλα δίνεται ο πίνακας μετάβασης και το διάγραμμα μετάβασης:

$A = \{0, 1, 2, 4, 7\} \leftarrow \text{αρχική}$

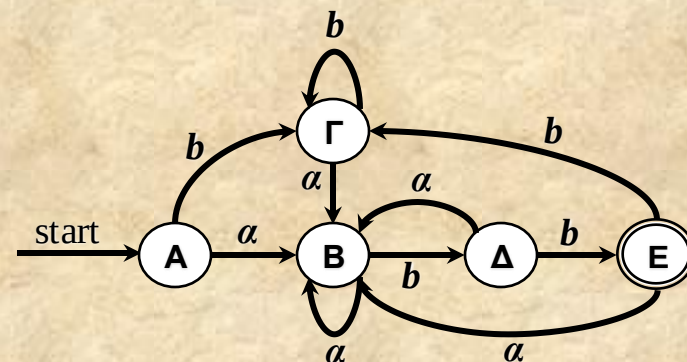
$B = \{1, 2, 3, 4, 6, 7, 8\}$

$\Gamma = \{1, 2, 4, 5, 6, 7\}$

$\Delta = \{1, 2, 4, 5, 6, 7, 9\}$

$E = \{1, 2, 4, 5, 6, 7, 10\} \leftarrow \text{τελική}$

Κατάσταση	Σύμβολο εισόδου	
	a	b
A	B	Γ
B	B	Δ
Γ	B	Γ
Δ	B	E
E	B	Γ

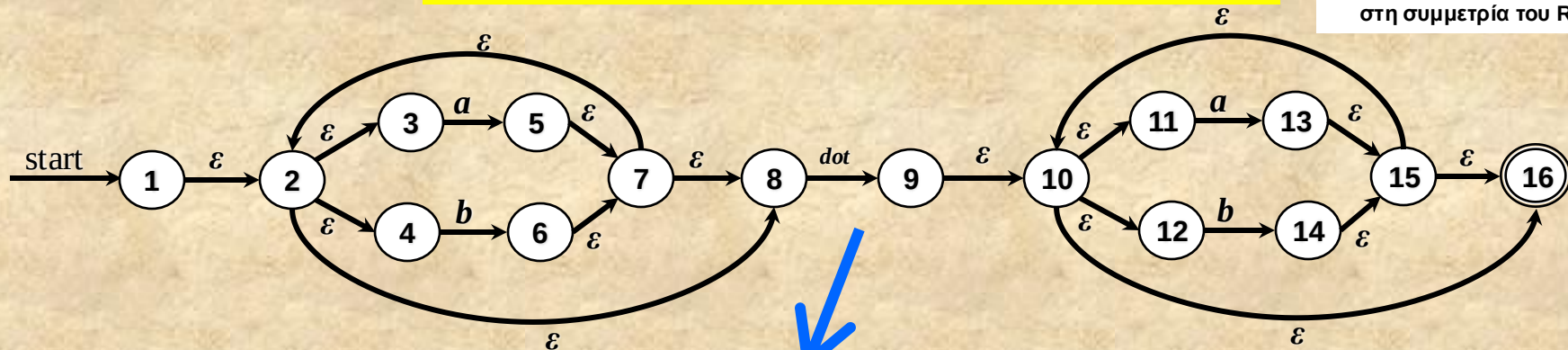




Αλγόριθμος μετατροπής $NFA \rightarrow DFA$ (11/11)

ΠΑΡΑΔΕΙΓΜΑ: μετατροπή του παρακάτω NFA, το οποίο δημιουργείται «μηχανιστικά» από την $(a|b)^*. (a|b)^*$, σε DFA

Η συμμετρία του αυτομάτου είναι προφανής και οφείλεται στη συμμετρία του RE



$A = \{1, 2, 3, 4, 8\} \leftarrow$ αρχική

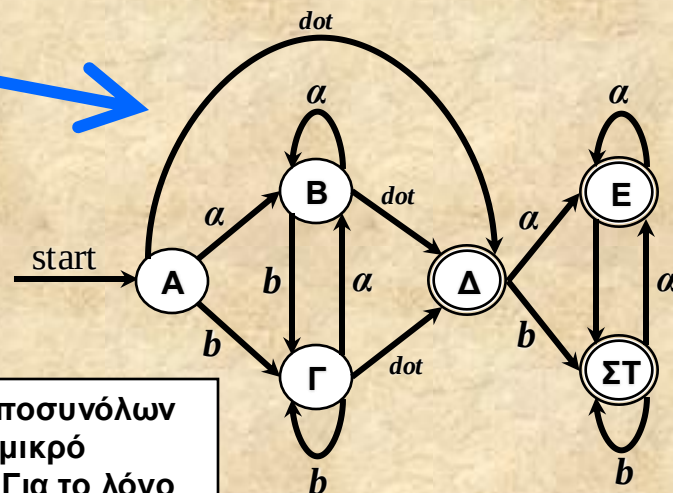
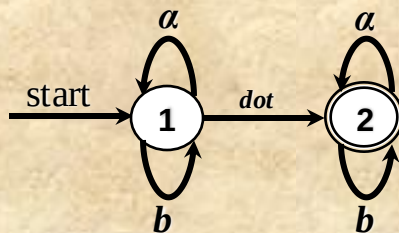
$\Delta = \{9, 10, 11, 12, 16\} \leftarrow$ τελική

$B = \{2, 3, 4, 5, 7, 8\}$

$E = \{10, 11, 12, 13, 15, 16\} \leftarrow$ τελική

$\Gamma = \{2, 3, 4, 6, 7, 8\}$

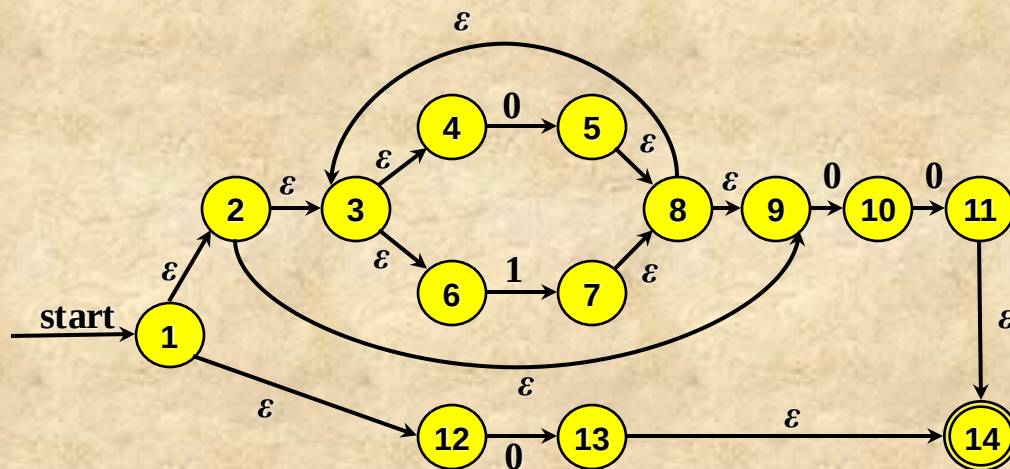
$\Sigma T = \{10, 11, 12, 14, 15, 16\} \leftarrow$ τελική



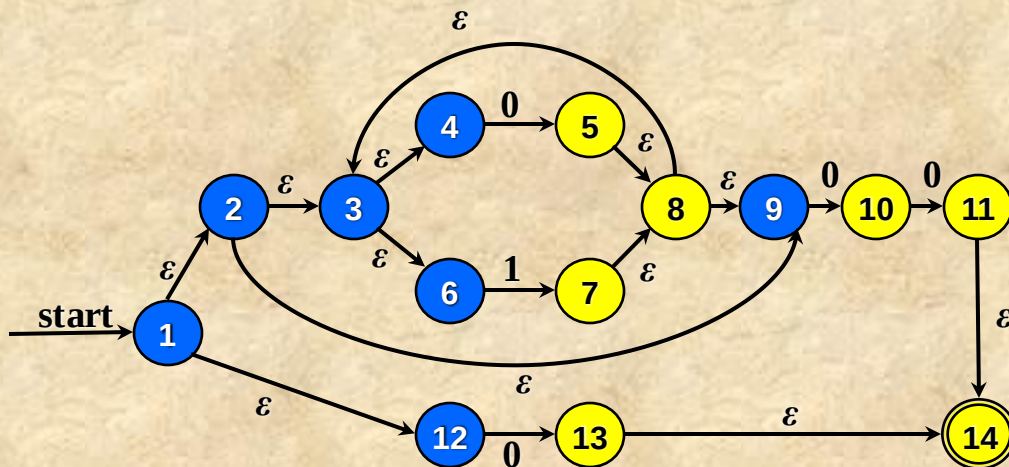
• Το αυτόματο στα δεξιά, που παράγεται από τον αλγόριθμο κατασκευής υποσυνόλων καταστάσεων για το παραπάνω NFA, αναγνωρίζει την ίδια γλώσσα με το μικρό αυτόματο πάνω. Είναι προφανές ότι το τελευταίο είναι πολύ απλούστερο. Για το λόγο αυτό υπάρχουν αλγόριθμοι ελαχιστοποίησης των καταστάσεων ενός DFA, καθώς τα «μηχανιστικά» παραγόμενα DFAs δεν είναι βέλτιστα.



Λεπτομερές παράδειγμα NFA $\rightarrow$ DFA (1/3)



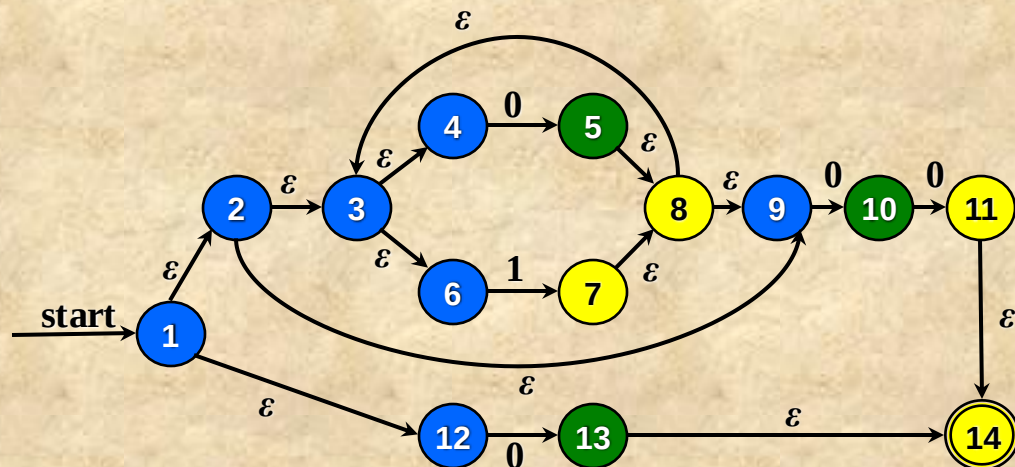
Αρχικό NFA



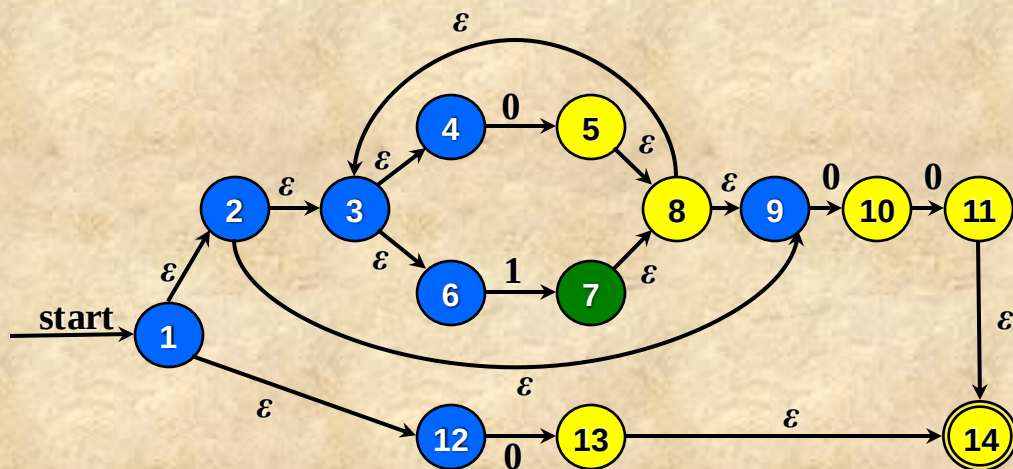
e-closure(s_1)



Λεπτομερές παράδειγμα NFA $\rightarrow$ DFA (2/3)



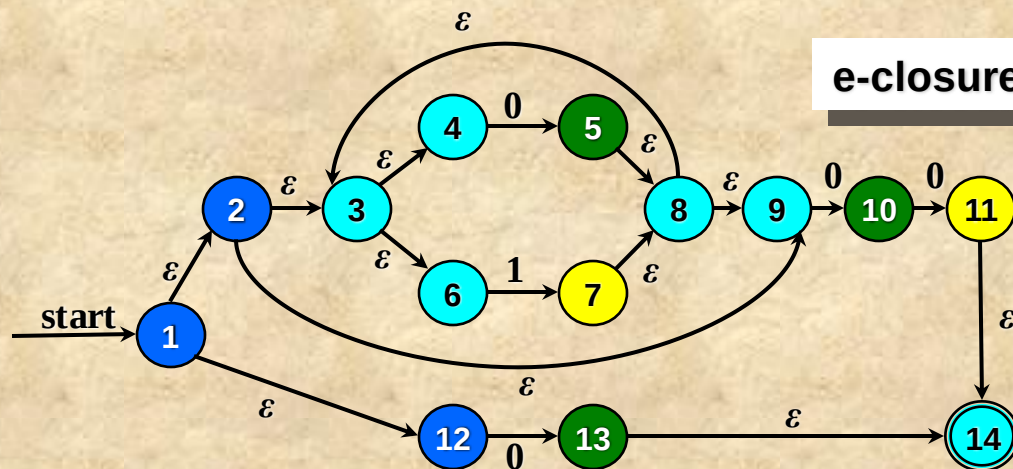
`move(e-closure( $s_1$ ), 0)`



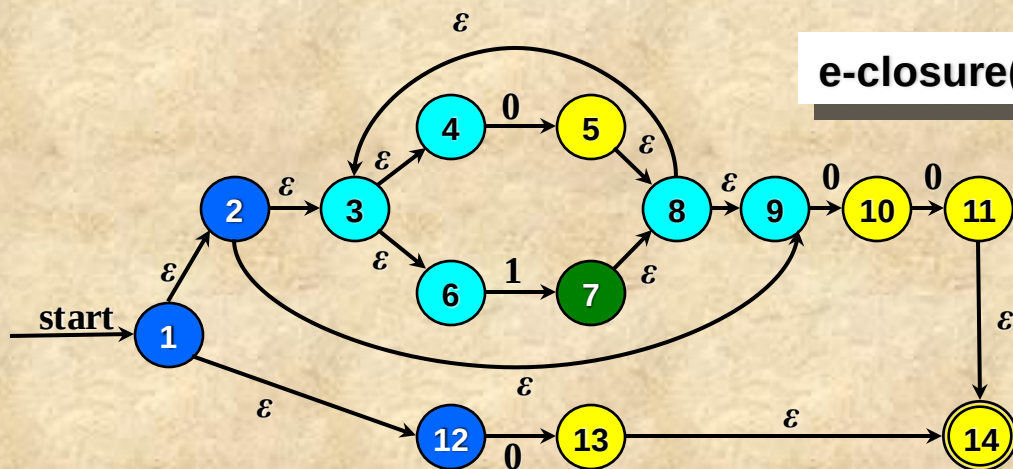
`move(e-closure( $s_1$ ), 1)`



Λεπτομερές παράδειγμα NFA $\rightarrow$ DFA (3/3)



$\text{e-closure}(\text{move}(\text{e-closure}(s_1), 0))$



$\text{e-closure}(\text{move}(\text{e-closure}(s_1), 1))$

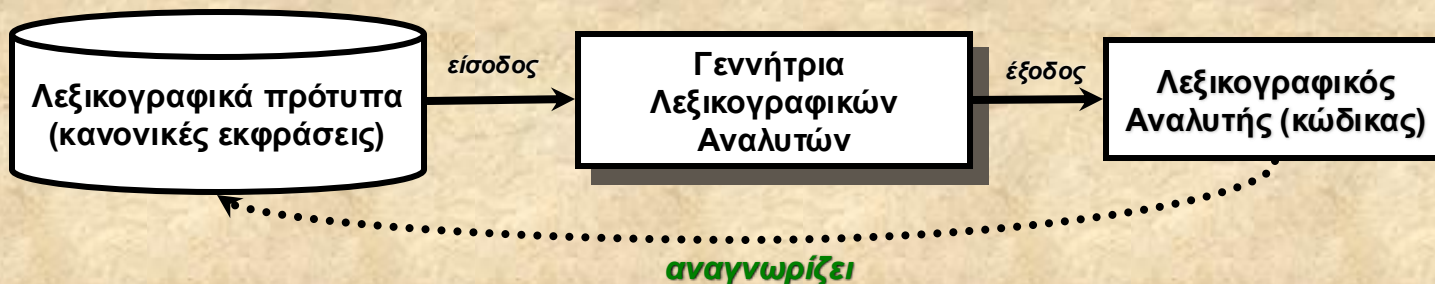


Περιεχόμενα

- Αιτιοκρατικά αυτόματα - DFA
- Αλγόριθμος μετατροπής NFA $\rightarrow$ DFA
- *Γεννήτριες λεξικογραφικών αναλυτών*
- Το πρόγραμμα Lex

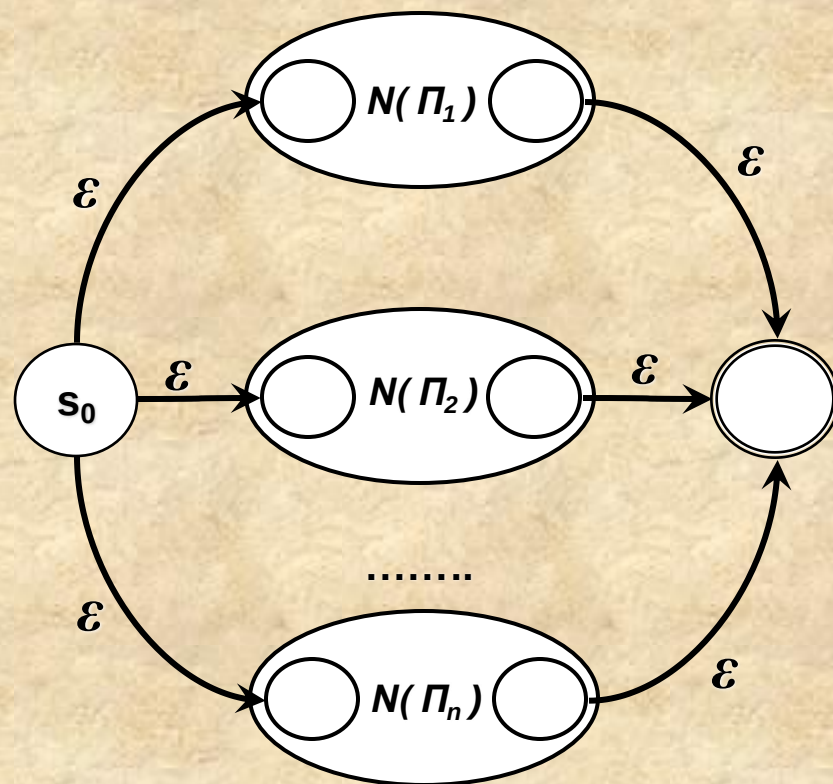
Γεννήτριες Λεξικογραφικών αναλυτών (1/3)

- Γενικά μία γεννήτρια λεξικογραφικών αναλυτών (*lexical analyzer generator*) είναι ένα πρόγραμμα το οποίο:
 - δέχεται ως είσοδο ένα σύνολο από **κανονικές εκφράσεις** που περιγράφουν λεξικογραφικά πρότυπα
 - παράγει ως έξοδο έναν **λεξικογραφικό αναλυτή** ο οποίος αναγνωρίζει το αντίστοιχο πρότυπο για κάθε ακολουθία συμβόλων εισόδου



Γεννήτριες Λεξικογραφικών αναλυτών (2/3)

- Οι γεννήτριες εφαρμόζουν τις τεχνικές που είδαμε για να παράγουν τελικά ένα DFA το οποίο αναγνωρίζει την ένωση των κανονικών γλωσσών των δεδομένων εκφράσεων - προτύπων
- Καθώς παρέχονται πολλές κανονικές εκφράσεις, μία για κάθε λεξικογραφικό πρότυπο π_i , μία τεχνική είναι η κατασκευή ενός συνδυασμένου NFA όπως φαίνεται δίπλα
- Άλλη τεχνική είναι η παραγωγή ανεξάρτητων DFAs τα οποία και εξομοιώνονται παράλληλα κατά την λεξικογραφική ανάλυση (όπως έχουμε δείξει)





Γεννήτριες Λεξικογραφικών αναλυτών (3/3)

- Θα δούμε μία πολύ διαδεδομένη και ισχυρή γεννήτρια, το πρόγραμμα *Lex*
 - Παράγει ολοκληρωμένους αναλυτές, δηλ. έτοιμο πρόγραμμα, που μπορεί να χρησιμοποιηθεί αυτούσιο στην κατασκευή ενός compiler
 - Επιτρέπει την συσχέτιση κανονικών εκφράσεων με blocks κώδικα
 - ◆ Μόλις αναγνωρίζεται ένα πρότυπο, καλείται και ο σχετικός κώδικας
 - Διαβάζει ένα αρχείο που περιέχει κανονικές εκφράσεις και τα αντίστοιχα blocks κώδικα και παράγει όλον τον κώδικα του λεξικογραφικού αναλυτή



Περιεχόμενα

- Αιτιοκρατικά αυτόματα - DFA
- Αλγόριθμος μετατροπής NFA $\rightarrow$ DFA
- Γεννήτριες λεξικογραφικών αναλυτών
- *Το πρόγραμμα Lex*



Το πρόγραμμα Lex (1/5)

- Θεωρήστε τη βασική συνάρτηση ***gettoken()*** την οποία τώρα την μετονομάζουμε σε ***yylex()***.
- Θεωρήστε ότι πρέπει να ορίζετε τις κανονικές εκφράσεις (λεξικογραφικά πρότυπα) για κάθε λεξικογραφικό στοιχείο της γλώσσας, π.χ.
 - *letter (letter | digit)** αναγνωριστικά
 - *digit+* σταθερά ακέραιος
 - *delimiter+* κενά
- Θεωρήστε ότι για κάθε έκφραση προσδιορίζετε το αιτιοκρατικό αυτόματο, καθώς και το συνολικό αυτόματο για όλες τις εκφράσεις μαζί.
- Θεωρήστε ότι βάσει του αυτομάτου, και χρησιμοποιώντας την τεχνική κατασκευής με κωδικοποίηση μεταβάσεων, υλοποιείτε «με το χέρι» έναν λεξικογραφικό αναλυτή.

Το πρόγραμμα Lex (2/5)

```
case 1:
    if (c == '=') return LE;
    Retract(c); return LT;
case 4:
    if (c == '=') return NE;
    Retract(c); return NOT;
case 15:
    if (isspace(c))
        { CheckLine(c); state = 15; }
    else
        Retract(c);
    break;
case 13:
    if (isalpha(c) || isdigit(c)) state = 13;
    else { Retract(c); return IDENTIFIER; }
    break;

default: assert(0); /* How did we get here? */
}

/* Append the present character to lexeme. */
lexeme[curr++] = c;
}
```

•Ο αναλυτής που φτιάχνουμε μπορείτε να θεωρήσετε ότι μοιάζει με αυτόν δίπλα, για την τεχνική state hard-coding.

•Όπως φαίνεται, ο μόνος κώδικας που δεν αφιερώνεται στην αναγνώριση προτύπων, είναι αυτός σε συμπαγές πλαίσιο, που καλείται αφού γίνει η αναγνώριση, επιστρέφοντας την τιμή με την οποία έχουμε επιλέξει να προσδιορίζουμε μοναδικά το token.

•Επιπλέον, υπάρχουν κάποιες εσωτερικές μεταβλητές του αναλυτή τις οποίες θα θέλαμε να τις έχουμε διαθέσιμες στο τμήμα που καλεί την `gettoken()`, όπως:

- `lineNo`
- `lexeme`
- `curr`

Ο Lex κατασκευάστηκε για να αυτοματοποιήσει αυτή την πολύ συχνά επαναλαμβανόμενη διαδικασία ως εξής:

•Περιγράφετε τις κανονικές εκφράσεις για κάθε πρότυπο P_i ενώ μαζί με αυτό παρέχετε και ένα block κώδικα που θα κληθεί αμέσως μετά την αναγνώριση του λεξικογραφικού προτύπου.

•Ο Lex συνθέτει την υλοποίηση της `gettoken()`, ονομαζόμενη τώρα `yylex()`, παράγοντας αυτόματα τον κώδικα αναγνώρισης, που αντιστοιχεί στο διακεκομμένο πλαίσιο, τοποθετώντας στα σημεία που ακριβώς έχει επιτελεστεί η αναγνώριση τα αντίστοιχα blocks που έχετε εισάγει. Σε κάθε τέτοιο block πρέπει να βάλετε την εντολή `return` με την επιθυμητή (για το token) τιμή, όπως και ο κώδικας που γράψαμε για τον δικό μας αναλυτή.



Το πρόγραμμα Lex (3/5)

```
%{
```

C ορισμοί σταθερών για κάθε τύπο token (π.χ. macros), καθώς και *C* κώδικας.

```
%}
```

Ορισμοί κανονικών εκφράσεων

```
delim
```

```
[ \t\n]
```

```
' ' '\t' '\n'
```

```
ws
```

```
{delim}+
```

```
letter
```

```
[a-zA-Z]
```

```
'a'|...'z'|'A'|...'Z'
```

```
digit
```

```
[0-9]
```

```
underscore
```

```
[_]
```

```
id
```

```
{letter} ({letter} | {digit} | {underscore}) *
```

```
number
```

```
{digit}+
```

```
%%
```

Κανόνες αναγνώρισης κανονικών εκφράσεων

```
{id}      { yylval = install(ytext); return ID; }
```

```
if        { return IF; }
```

```
"<"      { return LT; }
```

```
{number}  { yylval = atoi(ytext); return INTCONST; }
```

```
%%
```

C κώδικας (π.χ. συναρτήσεις που χρησιμοποιούνται στα παραπάνω blocks)

Στην μεταβλητή *yylval* αποθηκεύεται το χαρακτηριστικό γνώρισμα του token (π.χ. *symbol**, *int*, *char**, κλπ), ενώ ο τύπος του token επιστρέφεται με *return*.

Τα blocks του κώδικα που ορίζονται τοποθετούνται στην *yylex*, δηλ. την *gettoken*. Έτσι, το *return* ουσιαστικά είναι *return* της *yylex*.



Το πρόγραμμα Lex (4/5)

- Μέσα στα blocks φυσιολογικά έχετε τη δυνατότητα να χρησιμοποιήσετε ότι τύπο, μεταβλητή ή συνάρτηση έχει ήδη οριστεί στο πρώτο τμήμα (η σε `#include` που υπάρχει σε αυτό)
- ...καθώς και τα παρακάτω (μεταξύ άλλων) τα οποία εμπεριέχονται ήδη στο τμήμα του κώδικα που παράγεται αυτόματα από το Lex
 - ***yytext***, αντιστοιχεί στο *lexeme*, και είναι μεταβλητή ***char**** (για read-only σκοπούς), παρέχοντας τη λέξη που μόλις αναγνωρίστηκε για κάποιο πρότυπο
 - ***yyleng***, αντιστοιχεί στο *curr*, ***int***, και είναι ο αριθμός των χαρακτήρων που έχουν επιτυχώς αναγνωριστεί ως τμήμα του προτύπου
 - ***yylineno***, αντιστοιχεί στο *lineNo*, ***int***, η εκάστοτε γραμμή του αρχείου εισόδου (πρέπει να τη διαχειρίζεστε εσείς καθώς δεν λαμβάνει τιμές αυτομάτως)
- Περισσότερες λεπτομέρειες και παραδείγματα θα δοθούν στα φροντιστήρια. Προφανώς θα χρειαστεί να διαβάσετε πολύ καλά το manual του Lex.

Το πρόγραμμα Lex (5/5)

- Ένα ερώτημα που θα πρέπει να πλανάται είναι εάν έχει νόημα πλέον η κατασκευή ενός λεξικογραφικού αναλυτή από την αρχή
 - δεδομένης της ύπαρξης εργαλείων όπως ο Lex, για διάφορες γλώσσες προγραμματισμού
 - ◆ τα οποία επιπλέον προσφέρουν ιδιαίτερη ευελιξία ως προς τον τρόπο χρήσης και ενσωμάτωσης μέσα στο πρόγραμμά σας
 - Η απάντηση είναι μάλλον απλή
 - ◆ εάν θέλετε να φτιάξετε έναν πολύ εξειδικευμένο αναλυτή και δεν σας καλύπτουν Lex είδους εργαλεία
 - token reading with a timeout (network)
 - token patterns that change dynamically (natural language keywords?)
 - ◆ ή καθαρά για λόγους απόκτησης εμπειρίας (για να γνωρίζετε ακριβώς πως κατασκευάζεται)
 - ➔ η συμβουλή είναι εάν δεν συντρέχει άλλος λόγος «χρησιμοποιήστε έναν *lexical analyzer generator*» και θα κερδίστε και σε χρόνο και σε ποιότητα αποτελέσματος